Software Requirements Specification (IEEE 830)

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the Academic Data Pipeline and Analytics Dashboard.

1.2 Scope

The system will ingest, validate, and permanently store academic records from tabular data formats. The application acts as a collector tool that allows users to switch fluently between the software and Excel via import and export functions. The primary focus is tracking students from the department, with detailed evaluations intended for students in the Master's in Software Engineering program. The first priority is to deliver a Minimum Viable Product (MVP) focused exclusively on data upload. The system operates on an internal network, meaning user management and encryption are explicitly out of scope. Master's and Bachelor's theses are not recorded in the system.

1.3 Definitions, Acronyms, and Abbreviations

- **Matrikelnummer:** Unique student identification number.
- **ACID:** Atomicity, Consistency, Isolation, Durability. Strict database transaction rules.
- **WiSe / SoSe:** Wintersemester / Sommersemester.
- **TA / AT:** Absence codes indicating a student did not attend the exam.
- **NE:** Absence code indicating a sick note was submitted for the exam.

2. Overall Description

2.1 Product Perspective

The application will function on an internal network. The system will serve as a lightweight process to integrate data from individual exam sheets into a centralized database while maintaining Excel as a primary handling method.

2.2 User Characteristics

The system is expected to support fewer than 10 users. Users primarily consist of employees within the Software Engineering group who handle exam data.

2.3 Constraints

The system must support both English and German languages. The application must utilize standard, highly maintainable UI frameworks. Complex visual components will be excluded to prioritize ease of maintenance. Maintainability and data integrity serve as the primary quality constraints.

3. Specific Requirements

3.1 External Interfaces

- The frontend must display a clean, white UI theme.
- The user interface should offer customizable features, including color coding.
- The system must provide a drag-and-drop or copy-paste interface for tabular file uploads.
- The system must allow users to export data directly into an Excel sheet.
- The user interface must display the full name of a course when a user hovers over an abbreviated course name.

3.2 Functions

- **F.1 (Data Ingestion):** The system must accept tabular data uploads and immediately flag any incorrectness or data issues to the user.
- **F.2 (Exam Data Model):** Finalized exam records must capture the student's first name, last name, Matrikelnummer, study program, and semester.
- **F.3 (Absence Handling):** The system must automatically ignore the predefined absence codes TA, AT, and NE, ensuring they are not counted as valid exam results.
- **F.4 (Grade Formatting):** The system must store academic grades as simple floats or doubles.

- **F.5 (Grade Retention):** The system must permit multiple exam attempts for a single course, but it must strictly store only the best achieved grade.
- **F.6 (Acronym Handling):** The system must prompt the user to accept any new study program abbreviation or acronym encountered during an import. The system must apply accepted acronyms automatically to all future imports.
- **F.7 (Student Data Updates):** The system must display a warning if a user's name changes for an existing Matrikelnummer before updating the record. The system must visually flag study program changes in red.
- **F.8 (Course Differentiation):** The system must treat courses with identical names but different credit values (e.g., 8 credits vs. 6 credits) as entirely separate entities.
- **F.9 (Course Management):** The system must provide users with the ability to manually add new courses.
- **F.10 (Data Archiving):** The system must automatically hide students from the overview dashboard if they have registered no exam activity for four or more years.
- **F.11 (Analytics Filtering):** The system must provide analysis views that allow users to filter performance metrics by semester and by course.
- **F.12 (Analytics Metrics):** The system must calculate and display the percentage of 5.0 grades compared to the number of students who passed. The system must calculate the average grade strictly among the students who passed.
- **F.13 (Performance Tracking):** The analytics dashboard must track and visualize whether specific courses get easier or harder over time.
- **F.14 (Exploratory Analytics):** The system must map the distribution of marks relative to student countries by pairing application portal metadata with the imported exam data.

3.3 Performance Requirements

- System performance optimization is not an initial architectural driver.
- System performance must be calibrated for a concurrent user base of fewer than 10 users.

3.4 Logical Database Requirements

DB.1 (Constraint Enforcement): The database SHALL enforce data integrity natively at the schema level to provide a baseline security layer independent of application code.

- **DB.1.1:** The database SHALL utilize strict `CHECK` constraints for numeric boundaries (e.g., `CHECK (points BETWEEN 1.0 AND 5.0)`).
- **DB.1.2:** The database SHALL enforce strict increment rules for grading (e.g., explicitly allowing only 1.0, 1.3, 1.7, etc.) using PostgreSQL system-catalog constraints.

DB.2 (Reconciliation Monitoring): The system SHALL utilize scheduled background jobs to periodically scan the live database for data anomalies.

- **DB.2.1:** The reconciliation job SHALL identify and log integrity violations, specifically orphaned data where a foreign key points to a non-existent primary record.

DB.3 (Query Validation): The system SHALL utilize SQL linting tools (e.g., SQLMesh) configured with user-defined rules to validate database queries before execution.

3.5 Design Constraints

DC.1 (Development Operations): The development lifecycle SHALL be restricted to short-lived branches to ensure quality gates run on small, manageable code changes, facilitating easier reviews and reverts.

DC.2 (Dependency Management): The repository SHALL integrate automated dependency update tools (e.g., Renovate or Dependabot) within the CI/CD pipeline to continuously offload vulnerability patching and maintainability overhead.

DC.3 (Architecture Documentation): All architectural decisions and structural changes SHALL be documented using formal Architecture Decision Records (ADRs).

DC.4 (Definition of Done): The development team SHALL adhere to a strict Definition of Done (DoD) that incorporates all quality metrics defined in this specification. The DoD SHALL serve as the objective source of truth for all pull request review checklists.

3.6 Software System Attributes

3.6.1 Maintainability

MNT.1 (Complexity Thresholds): The codebase SHALL maintain a flat structure and adhere to strict cyclomatic and cognitive complexity limits per method to ensure single responsibility and readability.

- **MNT.1.1:** A static analysis tool (SonarQube) SHALL be integrated into the repository to track independent execution paths.
- **MNT.1.2:** The system SHALL trigger a soft-warning for any method with a complexity score >10.
- **MNT.1.3:** The CI pipeline SHALL block the merging of any method with a complexity score >15.

MNT.2 (Code Duplication Limits): The codebase SHALL maximize code re-use and modifiability by restricting duplicated code blocks.

- **MNT.2.1:** The duplication tracker (SonarQube) SHALL identify structural duplication regardless of variable renaming.
- **MNT.2.2:** The system SHALL trigger a soft-warning if code duplication exceeds 3% of the codebase.
- **MNT.2.3:** The CI pipeline SHALL block the merge if code duplication reaches or exceeds 5%.

MNT.3 (API Documentation Coverage): The backend SHALL maintain a 100% documentation ratio for all exposed endpoints.

- **MNT.3.1:** Documentation completeness SHALL be enforced automatically by an OpenAPI linter (e.g., Spectral) to ensure complete API understandability.

3.7 Verification and Validation (Quality Assurance Gates)

Note: While IEEE 830 focuses heavily on product requirements, process-driven testing gates are formalized here to define the exact acceptance criteria for system components.

QA.1 (Stage 1: Local Pre-Commit): All code changes SHALL pass local pre-commit hooks, including automated formatting, code linting, and unit tests, prior to being committed to the remote branch.

QA.2 (Stage 2: Pre-Merge Pull Request): All code proposed for merging SHALL pass a strict CI/CD gate requiring:

- A minimum of one peer review approval.
- Successful execution of all unit and integration tests.
- Passage of static analysis and maintainability gates (complexity and duplication metrics).
- Fulfillment of predefined test coverage minimums.

QA.3 (Stage 3: Staging): Prior to production release, the release candidate SHALL undergo automated and manual Quality Assurance in a staging environment.

- **QA.3.1:** New functionalities SHALL pass End-to-End (E2E) and functional tests.
- **QA.3.2:** Existing functionalities SHALL pass regression tests to ensure no degradation of service.

QA.4 (Stage 4: Production Monitoring): The system SHALL execute automated, scheduled tests against the real production dataset to monitor live operational integrity as defined in DB.2.